



**Tecnológico
de Monterrey**

04. Trabajando en R

Ciencia de datos para la toma de decisiones II

**Jorge
Juvenal
Campos Ferreira**

✉ juvenal.campos@tec.mx

Programa de la sesión de hoy

- Retomar la introducción a R y RStudio
- Entender los elementos más sencillos del lenguaje
- Ver las funciones principales del tidyverse
- Abrir archivos con datos
- Hacer un ejemplo sencillo de manejo de datos

Personalización

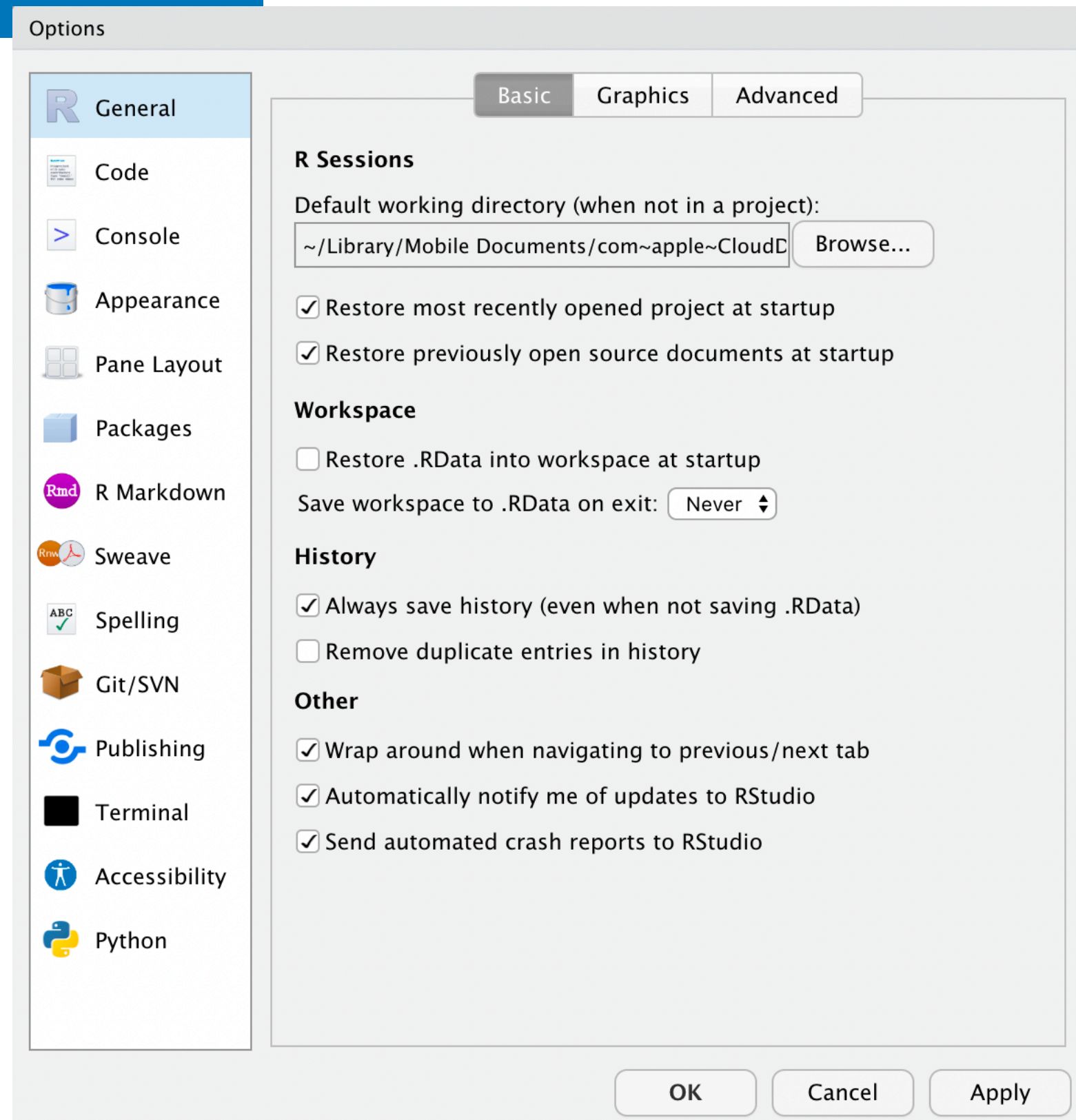


Para personalizar RStudio
vamos a

Tools > Global Options

Podemos configurar:

- como se visualiza el código,
- los colores de las ventanas,
- el tamaño y fuente de las letras,
- el espacio a ocupar del código,
- las cuentas para publicar resultados, etc.





Archivos nativos de R

Al trabajar con R y RStudio, generamos cuatro tipos de archivos nativos de R, estos son los siguientes:

- *.Rproj,
- *.RData,
- *.R y
- *.rds.

Los scripts son aquellos archivos que terminan en *.r

Objetos y funciones



- En R, todo lo que existe es un objeto.
- En R, todo lo que ocurre es una función.

Objetos y funciones



Los **objetos** son el lugar de la memoria en el cual vamos a guardar información.

Podemos crear nuestros **objetos** o podemos tomar **objetos** hechos por alguna librería.

Los **objetos** son sujetos de ser afectados por las **funciones**.

Las **funciones** son las acciones que le vamos a aplicar a un objeto para obtener un resultado, el cual va a ser un objeto.

Los **objetos** (y las **funciones**) se guardan en el ambiente. El ambiente es el lugar donde se almacenan los **objetos** de la sesión.

Funciones



Las **funciones** son las **acciones** que vamos a realizar sobre los **objetos**. Estas pueden ya estar precargadas de las **librerías base** o pueden provenir de **librerías** descargadas de manera externa.

Las funciones se llaman con la siguiente sintaxis:

```
nombre_funcion(argumento_1 = "argumento_1",  
                argumento_2 = "argumento_2",  
                ... ,  
                argumento_n = "argumento_n")
```

Los **argumentos** son como palanquitas a las que hay que moverle para que las funciones funcionen de manera adecuada.

Funciones



Las **funciones** tienen nombre y apellido. Muchas veces las vamos a encontrar en la literatura de la manera que sigue:

```
dplyr::filter()  
sf::read_sf()  
base::sum()  
leaflet::leaflet()
```

En este caso, lo que va a la izquierda de los **dos-dos puntos** es la **librería o paquetería** (apellido) de la cual provienen, mientras que lo que va al lado derecho es el nombre de la función.

Si llamamos la **librería** de origen, meter el apellido ya no va a ser necesario. :3. Si no se le pone apellido, es que proviene de **base**

Ambiente/entorno global

- El ambiente global en RStudio es el espacio de trabajo donde R guarda todos los objetos que se crean en la sesión
- Aquí viven los objetos que creamos con la `<-`
- En la sesión, vamos a poder acceder a los objetos creados en este ambiente.

Librerías



★ Definición

Las librerías son un conjunto de objetos y funciones programados por terceros, que podemos instalar en nuestra sesión de R para potenciar las funciones que podemos realizar.

Instalación

Las instalamos en la sección del **Visualizador**, en la sección de “*Packages*” > *Install* y escribimos la librería que queremos instalar. Otra opción es con la función `install.packages("librería_a_instalar")`

Instalar librerías



Opción 1

Install Packages

Install from: [? Configuring Repositories](#)

Repository (CRAN)

Packages (separate multiple with space or comma):

tidyverse, plotly, sf, scales

scales Library:

/Library/Frameworks/R.framework/Versions/4.0/Resources/librar

☒ Install dependencies

Install Cancel

Opción 2

```
> install.packages(c("tidyverse",  
  "plotly", "sf", "scales"))
```

Instalar librerías



★ Instalar librerías



Llamar librerías



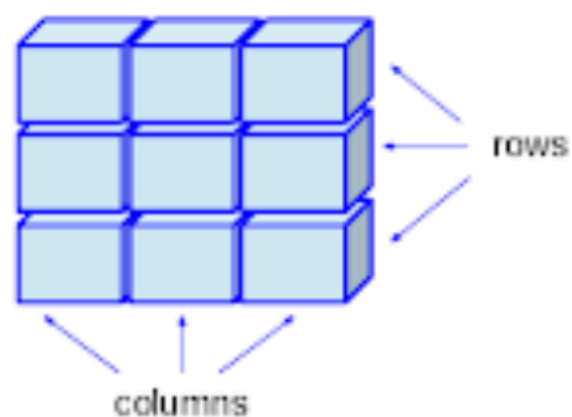
Estructura de Datos



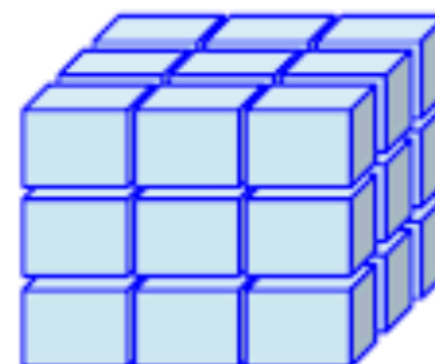
Vector



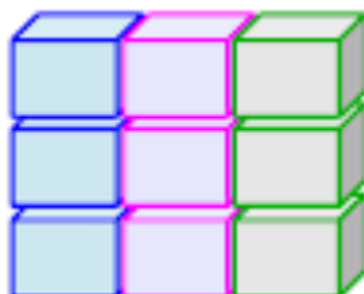
Matrix



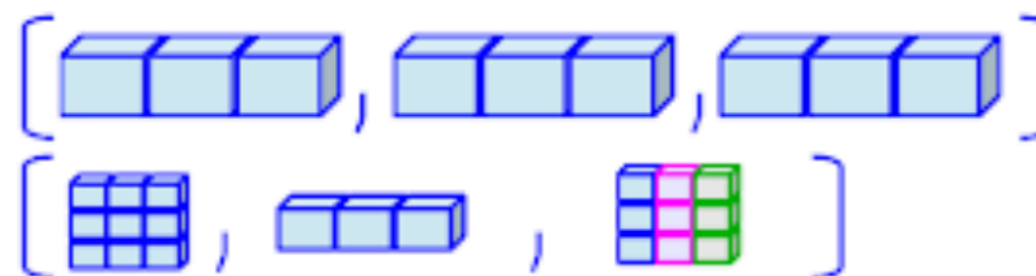
Array



Data Frame
(Table)



Lists

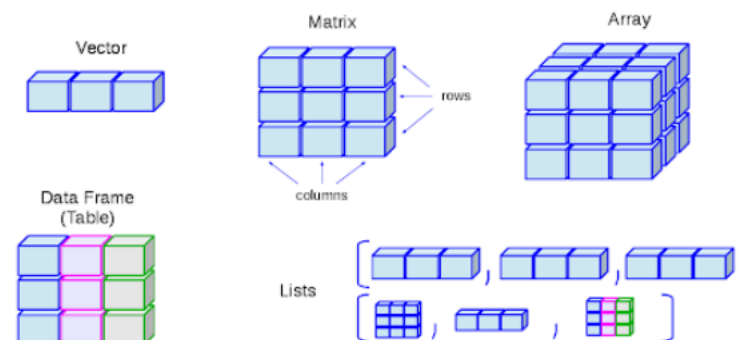


Estructura de Datos



Los datos suelen agruparse en estructuras básicas de datos:

1. **Escalares**, vectores de 1×1
2. **Vectores**, arreglos de $n \times 1$ (n renglones y 1 columna)
3. **Matrices**, arreglos cuadrados de $n \times n$ (n renglones y n columnas). Un único tipo de dato.
4. **Arrays**, matrices de matrices multidimensionales $n \times n \times \dots \times n$.
5. **DataFrames**, arreglos cuadrados de $n \times n$ (n renglones y n columnas). Múltiples tipos de dato.
6. **Listas**. Contenedores de cualquier cosa.



Estructura de Datos



Se pueden generar o extraer de muchas formas. Las mas sencillas son las siguientes:

Vectores: Usamos la función combine, **c()**.

```
vector_1 <- c(1,2,3,4,5)
```

Matrices: Usamos la función **matrix()** y le metemos como ingredientes para esa matriz vectores y argumentos como *ncol* o *nrow* para definir las dimensiones.

```
vector_1 <- c(1,2,3,4,5)
vector_2 <- 10:14
mtx <- matrix(c(vector_1, vector_2), ncol = 2)
```

Estructura de Datos



Se pueden generar o extraer de muchas formas. Las mas sencillas son las siguientes:

Dataframes: Podemos construir dataframes con la función **data.frame()** o con la función **tibble::tibble()**, y le pasamos como ingredientes vectores del mismo tamaño para que sirvan de columnas.

```
df <- data.frame(vector_1,  
                  vector_2,  
                  letras = c("a", "b", "c", "d", "e"))
```

Este es el tipo de estructura más usado; cuando leemos datos de excel, por ejemplo, nos va a construir automáticamente un DF.

Estructura de Datos



Se pueden generar o extraer de muchas formas. Las mas sencillas son las siguientes:

Listas: Podemos construir listas con la función **list()**. Como acá le podemos meter dentro la estructura de datos que sea, le metemos como ingredientes lo que sea.

```
lista <- list(df, # Le metemos un df
             mtx, # le metemos una matriz
             vector_2, # Le metemos un vector
             vector_1, # Le metemos otro vector
             mtcars, # Le metemos otro df pre-construido
             list(df, vector_2), # Le podemos meter listas
             sum() # Le podemos meter funciones
             )
```



Tipos de datos

(vectores atómicos)

Vectores atómicos



Los tipos de datos que podemos almacenar dentro de nuestras estructuras de datos son los siguientes:

Básicos:

- **Character**, para almacenar texto.
- **Numeric**, para almacenar números.
- **Logical**, para almacenar valores lógicos (TRUE/FALSE o T/F).

Compuestos:

- **Factor**, para almacenar datos categóricos.
- **Date**, para almacenar fechas.



Vectores atómicos

- **Character**, para almacenar texto.

```
character <- c("Hola",  
              "Adios",  
              "Buenas tardes",  
              "Buenas Noches")
```

```
character_2 <- c("1", "2",  
                "tres", "cuatro", "5")
```

```
macondo <- c("Muchos años después, frente al pelotón de  
fusilamiento, el coronel Aureliano Buendía había de recordar  
aquella tarde remota en que su padre lo llevó a conocer el hielo.  
Macondo era entonces una aldea de veinte casas de barro y  
cañabrava construidas a la orilla de un río de aguas diáfanas  
que se precipitaban por un lecho de piedras pulidas, blancas y  
enormes como huevos prehistóricos.")
```




Vectores atómicos

- **Numeric**, para almacenar números.

```
# Numéricos
enteros <- c(1L, 2L, 3L, 4L) # Enteros
reales <- c(1,2,3,4) # reales
reales_2 <- c(1e6, 3.1416, 2) # reales con notación científica
```

```
# Operaciones ----
```

```
# Suma
```

```
c(1,3,4,5) + 5 # [1] 6 8 9 10
```

```
# Resta
```

```
c(1,3,4,5) - 5 # [1] -4 -2 -1 0
```

```
# Multiplicación:
```

```
c(1,3,4,5) * 5 # [1] 5 15 20 25
```

```
# División:
```

```
c(1,3,4,5) / 2 # [1] 0.5 1.5 2.0 2.5
```

```
# Exponenciación:
```

```
c(1,3,4,5) ^ 2 # [1] 1 9 16 25
```

```
# Secuencias ----
```

```
1:100 # Numeros del uno al 100, de uno en uno
```

```
seq(from = 0, to = 100, by = 2)
```

```
# Numeros del cero al 100, de dos en dos
```



Vectores atómicos

- **Logical**, para almacenar valores lógicos (TRUE/FALSE o T/F).

```
# VALORES LÓGICOS
logical <- c(TRUE, TRUE, TRUE, TRUE, FALSE)
logical_2 <- c(T, T, T, T, F)
# Son lo mismo uno y otro
```

```
# Como resultado de comparaciones:
c(1,2,3,4,5) < 3 # [1] TRUE TRUE FALSE FALSE FALSE
c("a", "b", "c") == "c" # [1] FALSE FALSE TRUE
c("Morelos", "Zacatecas", "Aguascalientes", "CDMX") %in%
  c("Morelos", "Zacatecas") # [1] TRUE TRUE FALSE FALSE
```



Vectores atómicos

- **Factor**, para almacenar datos categóricos.

```
# PASO 1: GENERO UN VECTOR NORMAL
```

```
partidos_ganadores <- c("PAN",  
                        "PRI",  
                        "MORENA",  
                        "MORENA",  
                        "PRI",  
                        "MORENA",  
                        "MORENA",  
                        "MORENA")
```

```
# PASO 2: LO TRANSFORMO A CATEGÓRICO.
```

```
partidos_con_categorias <- factor(partidos_ganadores,  
                                  levels = c("PAN", "PRI", "MORENA"),  
                                  ordered = F)
```

```
> partidos_con_categorias
```

```
[1] PAN    PRI    MORENA MORENA PRI    MORENA MORENA MORENA
```

```
Levels: PAN PRI MORENA
```



Vectores atómicos

- **Factor**, para almacenar datos categóricos.

```
# Generamos el vector de texto
tamanios_playera <-
  c("mediano", "grande", "chico", "chico",
    "mediano", "grande", "grande", "mediano",
    "grande", "chico", "chico", "mediano")

# Sobreescribimos
tamanios_playera <- factor(tamanios_playera,
  levels = c("grande", "mediano", "chico"),
  ordered = T)

tamanios_playera
```

```
[1] mediano grande chico chico mediano grande grande mediano
[9] grande chico chico mediano
Levels: grande < mediano < chico
```

Vectores atómicos



- **Date**, para almacenar fechas.

```
fecha <- Sys.Date()  
fecha  
  
fechas_2 <- as.Date(c("2020-02-02",  
                      "2019-03-04",  
                      "1991-07-08"),  
                  format = "%Y-%m-%d")  
fechas_2
```

```
> fecha <- Sys.Date()  
> fecha  
[1] "2021-06-23"  
> fechas_2 <- as.Date(c("2020-02-02",  
+                        "2019-03-04",  
+                        "1991-07-08"),  
+                        format = "%Y-%m-%d")  
> fechas_2  
[1] "2020-02-02" "2019-03-04" "1991-07-08"
```

Función **class(x)**



La función **class(x)** es la función con la cual podemos saber qué tipo de dato tiene el objeto "x".

```
# Función class(x)
class(fechas_2) # [1] "Date"
class(partidos_con_categorias) # [1] "factor"
class(tamanios_playera) # [1] "ordered" "factor"
class(logical_2) # [1] "logical"
class(vector_1) # [1] "numeric"
class(character_2) # [1] "character"
```


Función **length(x)**



La función **length(x)** nos permite saber el tamaño (# de elementos) que contiene un vector.

```
# Función length(x)
length(fechas_2) # [1] 3
length(partidos_con_categorias) # 8
length(tamanios_playera) # 12
length(logical_2) # [1] 5
length(vector_1) # 5
length(character_2) # 5
```

Coerción.



Supongamos que mezclamos dos o mas tipos de datos:

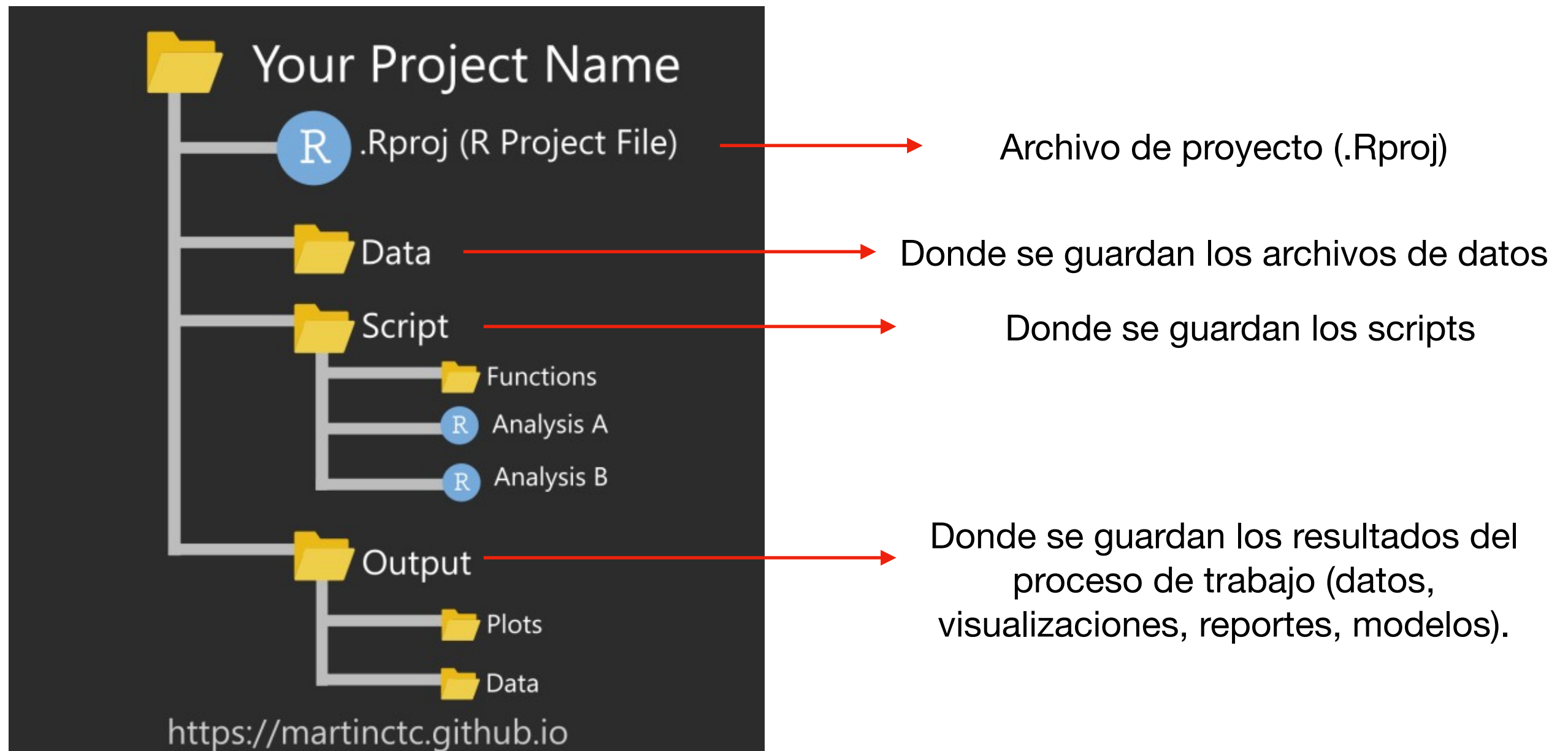
```
# Tres tipos de datos combinados  
c("Hola", TRUE, 1)
```

Como R no entiende que tipo de dato es el vector, y como necesita que sea de un solo tipo, lo va a transformar al tipo más simple: en este caso, todo lo va a hacer (coercionar) a TEXTO:

```
> c("Hola", TRUE, 1)  
[1] "Hola" "TRUE" "1"
```

Tenemos que tener un solo tipo de datos; si no se cumple esto, R va a transformarlos por nosotros.

Configuración de un proyecto básico de R



Estructura (sugerida) de un código o script

1. Configuración
2. Carga de librerías
3. Carga de parámetros y funciones propias
4. Carga de datos
5. Limpieza de datos
6. Análisis exploratorio de datos
7. Generación de funciones
8. Generación y pruebas de modelos
9. Generación de gráficas
10. Guardado de archivos

```
# -----  
# Autor: Juvenal Campos  
# Fecha: 2025-08-18  
# Proyecto: Ejemplo para el Tec de Monterrey  
# Descripción: Script dummy que ilustra la estructura básica de un flujo en R.  
# -----  
  
# 1) Configuración -----  
options(digits = 3)  
set.seed(42)  
  
# 2) Carga de librerías -----  
library(dplyr)  
library(readr)  
library(ggplot2)  
  
# 3) Parámetros y funciones propias -----  
params <- list(  
  input = "data/input.csv",      # <- reemplaza si tienes archivo  
  output_dir = "out",  
  target = "mpg"  
)  
paleta_colores <- c("#6950d8", "#3CEAFA", "#00b783",  
                    "#ff6260", "#ffaf84", "#ffbd41")  
if (!dir.exists(params$output_dir)) dir.create(params$output_dir, recursive = TRUE)  
  
clean_names <- function(x){  
  names(x) <- tolower(gsub("[^a-z0-9_]+", "_", names(x)))  
  x  
}  
rmse <- function(y, yhat) sqrt(mean((y - yhat)^2))  
  
# 4) Carga de datos -----  
# df <- read_csv(params$input)      # <- real  
# Dummy para ilustrar:  
df <- clean_names(mtcars)
```

Estructura (sugerida) de un código o script

1. Configuración
2. Carga de librerías
3. Carga de parámetros y funciones propias
4. Carga de datos
- 5. Limpieza de datos**
- 6. Análisis exploratorio de datos**
- 7. Generación de funciones**
- 8. Generación y pruebas de modelos**
- 9. Generación de gráficas**
- 10. Guardado de archivos**

```
# 5) Limpieza de datos -----
df <- df %>%
  mutate(across(everything(), \(v) v)) %>%
  filter(!is.na(.data[[params$target]]))

# 6) Análisis exploratorio (EDA) -----
print(summary(df))
print(df %>% summarise(n = n(), p = ncol(.)))

# 7) Generación de funciones -----
# (ya definimos `rmse()` arriba como ejemplo)

# 8) Modelado (dummy) -----
mod <- lm(mpg ~ wt + hp, data = df)
pred <- predict(mod, df)
cat("RMSE:", rmse(df$mpg, pred), "\n")

# 9) Gráficas -----
p <- ggplot(df, aes(wt, mpg)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE)

ggsave(file.path(params$output_dir, "scatter.png"), p,
        width = 5, height = 4, dpi = 150)

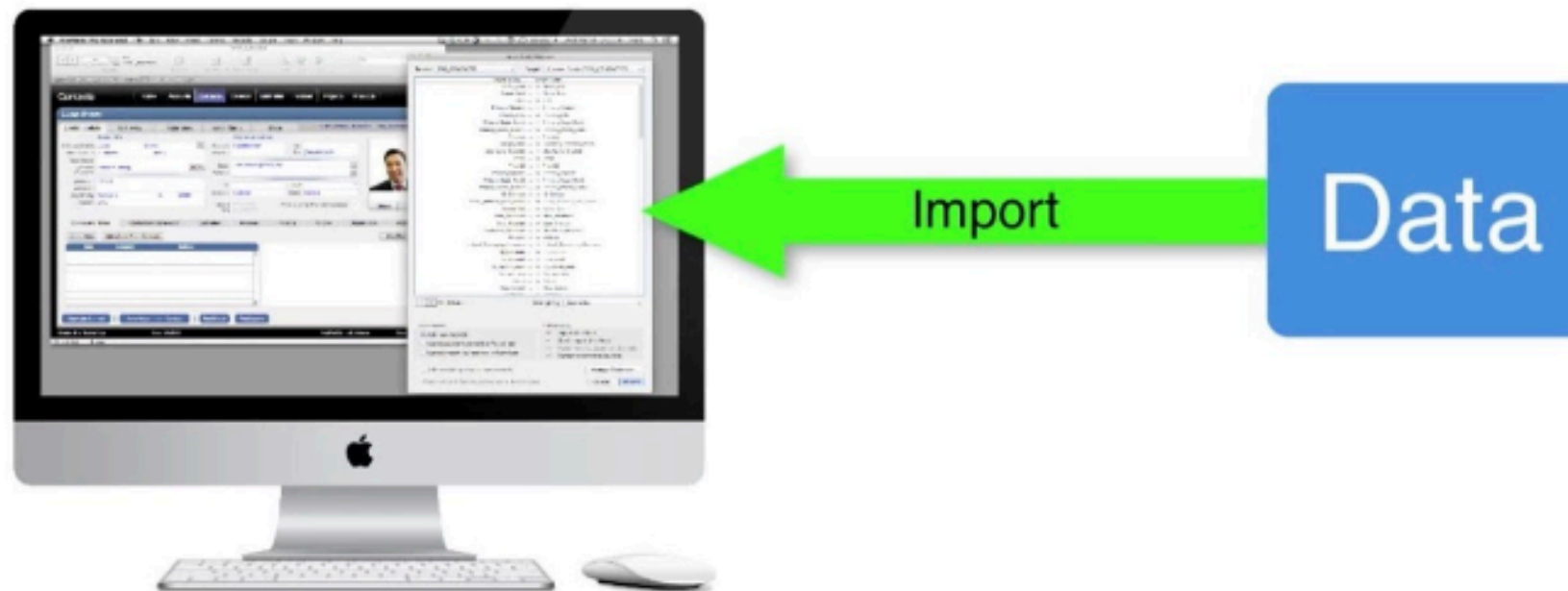
# 10) Guardado de archivos -----
write_csv(df, file.path(params$output_dir, "data_clean.csv"))
saveRDS(mod, file.path(params$output_dir, "model.rds"))
cat("Archivos guardados en:", normalizePath(params$output_dir), "\n")
```

Importar de datos

Importar datos

Cargar datos de un archivo externo a nuestra sesión de R.

IMPORTING DATA



Directorio de trabajo

El directorio de trabajo es la carpeta de la computadora en la cual vamos a estar trabajando.

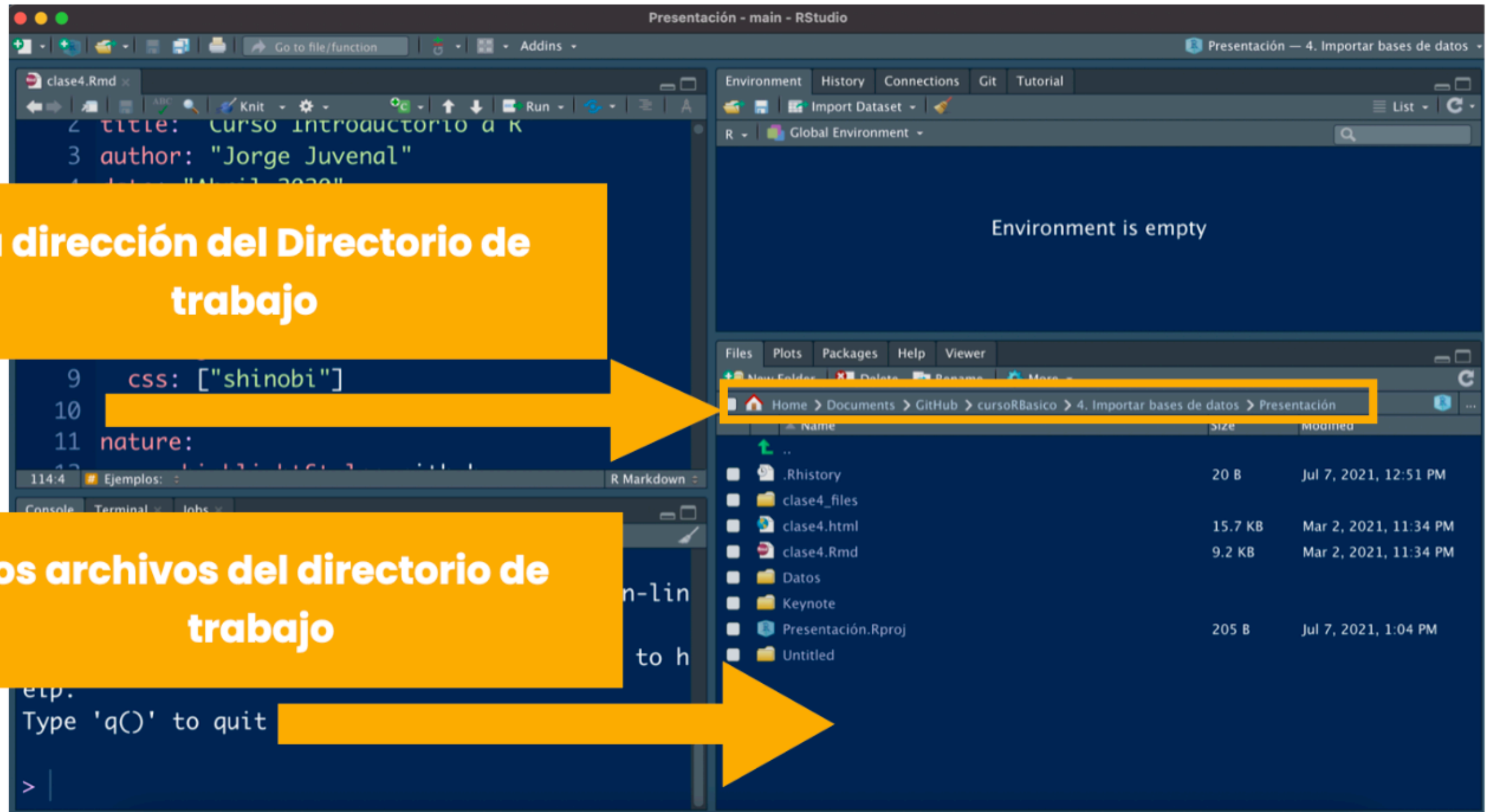
En esta carpeta deben ir (idealmente) todos los archivos que contienen los datos que vamos a utilizar.

En esta carpeta también se van a guardar los archivos que vamos a generar derivados de nuestro análisis (visualizaciones, otras bases de datos, etc.).

Directorio de trabajo

La dirección del Directorio de trabajo

Los archivos del directorio de trabajo



Ubicaciones de un archivo

Para acceder al contenido de un archivo, el programa debe saber donde se ubica.

Podemos acceder a archivos que se encuentren en:

A) Tu computadora

B) En internet

Archivos de internet

Si el archivo se encuentra en internet, una de dos:

a) Lo leemos directamente desde internet.

Hay archivos que se pueden leer desde internet sin necesidad de descargarlos a la compu. Esto depende de la función que usemos, del tipo de archivo y de los permisos de acceso (generalmente solo se puede con archivos de texto plano o archivos únicos).

b) Lo descargamos a nuestra computadora.

Lo descargas y ya. Lo lees desde tu compu. Esto pasa generalmente con excedes, archivos shapefile o archivos *.dta.

Archivos locales

Son los archivos que viven en nuestra computadora.

Para acceder a ellos, necesitamos:

- 1) **La función de R correcta** (y la librería que la contiene).
- 2) **La ubicación** global o la ubicación local de nuestro archivo.

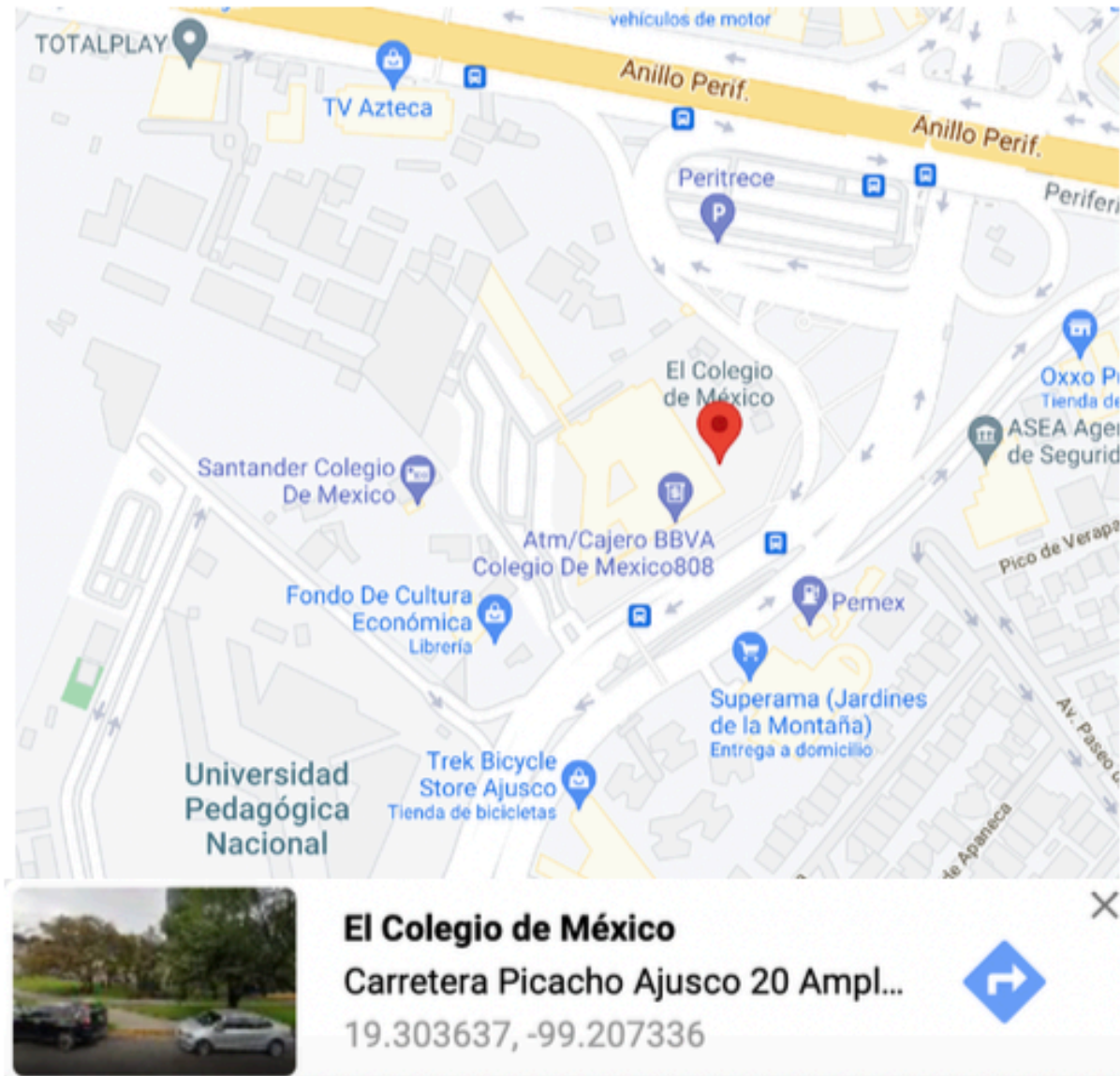
Ruta Global y Ruta Relativa

La ruta global y la ruta local son las ubicaciones de un archivo dentro de nuestra computadora.

La ruta global es la ruta absoluta y única de un archivo en la computadora, y es una forma de identificar el espacio en la memoria que ocupa dicho archivo.

La ruta local es la ruta relativa a otra ubicación en nuestra computadora.

Ruta Global y Ruta Relativa



Ruta absoluta



Ruta relativa

Ruta Global y Ruta Relativa

Las ventajas de usar la ruta absoluta es que siempre va a funcionar en tu computadora. Y ya.

La ventaja de la ruta relativa es que la puedes generar de forma en que **funcione en tu computadora y en las computadoras de otras personas, lo cual mejora la *replicabilidad* de tu análisis y permite compartirlo con otras personas.**

Creando rutas relativas

Hay 3 formas de crear rutas relativas en R:

- 1) Con la función `setwd()`**
- 2) Con la función `here::here()`**
- 3) Utilizando archivos de proyecto de RStudio.**

La que más recomendamos es la opción 3.

Archivo de proyecto (.Rproj)



=



Archivos más utilizados

Los archivos más utilizados para guardar datos son los siguientes:



Información tabular



Objetos de R



Información geográfica



Otros formatos

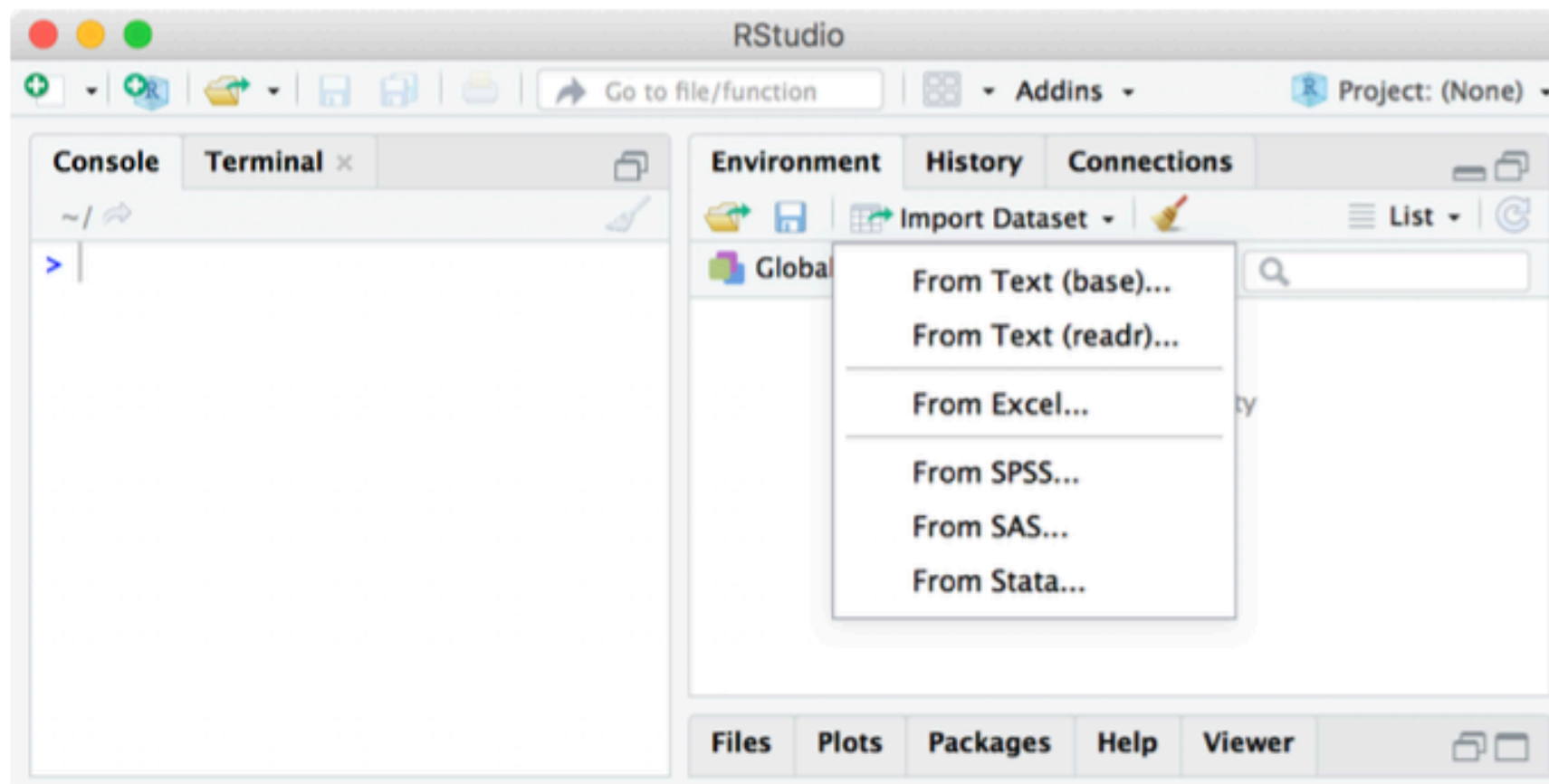
Librerías para importar datos

- `library(readr)`: Archivos texto plano.
- `library(readxl)`: Para excel
- `library(haven)`: Para archivos de Stata, SAS o SPSS
- `library(foreign)`: Para archivos de Stata, SAS o SPSS
- `library(sf)`: Leer archivos geográficos.

Formas de importar datos

Podemos importar archivos de dos maneras:

- 1) Con el **asistente**.*
- 2) Con **código** puro.

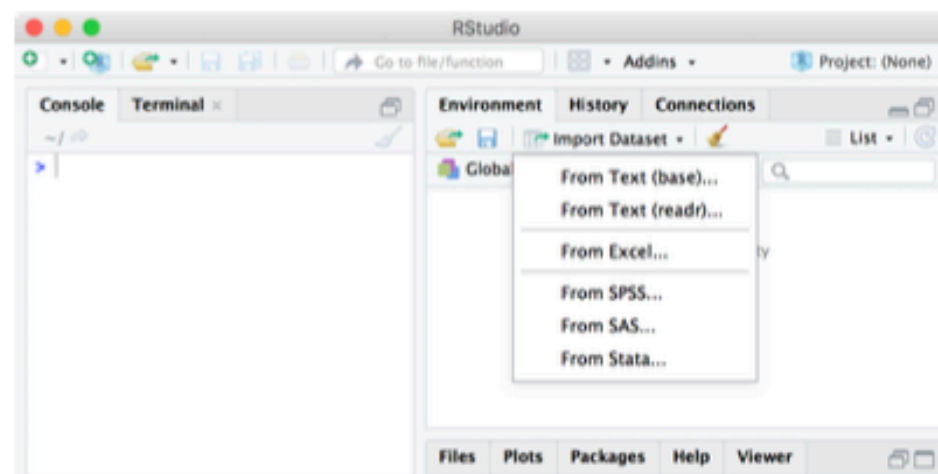


*Sólo para archivos de texto plano, excel, SPSS, Stata y SAS

Formas de importar datos

Método 1: ASISTENTE DE RSTUDIO.

- También puede hacerse el mismo procedimiento a través del Menú:
 - File > Import Dataset > From__ > Browse > Archivo > Import
- **Ventajas:**
 - **Acceso directo al archivo** y R genera la ruta y función correctas
 - Permite **agregar parámetros simples** para la importación (v.g. sheets, skip)
 - **Visualización** de cómo será importado el conjunto de datos
- **Desventajas:**
 - Supone **pasos extra** (copiar y pegar el código) para replicabilidad
 - No permite agregar todos los parámetros (v.g. stringsAsFactors, encoding)



Formas de importar datos

Método 2: SÓLO CON CÓDIGO

- Generando el código directamente:
 - **Requiere cargar las librerías específicas y establecer la ruta del archivo**
- **Ventajas:**
 - Puedes **importar múltiples archivos a la vez** y de todos los formatos posibles
 - **Agregar argumentos específicos** para una importación más sofisticada
 - **Incluir** importación de archivos **dentro de loops o funciones**
 - Si te sabes la función, es **más rápido de escribir**.
- **Desventajas:**
 - Requiere saberte la función exacta
 - Hay que **conocer como es el archivo por dentro**
 - **Pueden ocurrir errores** de los que nos hubiéramos percatado con el otro método.

Problemas de importación

1. Encodings raros.

El **encoding** es el conjunto de símbolos utilizados por un programa para desplegar texto.

Tu sesión de RStudio trabaja por default con un encoding que **podría ser distinto** al que utilizan en otras oficinas/empresas, al que utilizan otros programas o al que usan en otros países.

Por esto mismo, **puede ser necesario el modificar el encoding** utilizado al abrir un archivo o base de datos.

Problemas de importación

2. Errores humanos.

Los más comunes son:

- 1) **Escribiste mal la ruta** del archivo y R no lo encuentra.
- 2) **No conoces bien el archivo** y no te saltaste las líneas necesarias para que se abriera bien.
- 3) **No especificaste la hoja correcta de excel** y te abrió la primera
- 4) No especificaste el **encoding correcto**.

Gracias